



تمرینات گیت و پایتون هفته دوم

تمرین ۱: نصب **Git** ، ایجاد حساب **GitHub** و دریافت یک ریپازیتوری روی سیستم

در این تمرین هدف این است که **Git** را نصب کنید، حساب **GitHub** بسازید و یک ریپازیتوری را از **GitHub** روی سیستم خود دریافت کنید.

بخش ۱: بررسی نصب **Git**

ابتدا مطمئن شوید **Git** روی سیستم شما نصب است.

در ترمینال یا **Git Bash** دستور زیر را اجرا کنید:

```
git --version
```

اگر نسخه **Git** نمایش داده شد، نصب **Git** درست انجام شده است.

بخش ۲: ساخت حساب **GitHub**

اگر حساب **GitHub** ندارید، یک حساب کاربری در **GitHub** بسازید و ایمیل خود را تأیید کنید.

بخش ۳: ساخت ریپازیتوری در GitHub

در GitHub یک ریپازیتوری جدید بسازید.

مشخصات ریپازیتوری:

Repository name: karvand-manager

گزینه Initialize with README را فعال نکنید.

بخش ۴: دریافت ریپازیتوری روی سیستم

لینک ریپازیتوری خود را کپی کنید و با دستور زیر آن را روی سیستم Clone کنید:

```
git clone https://github.com/USERNAME/karvand-manager.git
```

سپس وارد پوشه پروژه شوید:

```
cd karvand-manager
```

بخش ۵: تنظیم اطلاعات Git فقط برای همین پروژه

در داخل پوشه پروژه، نام و ایمیل خود را به صورت local تنظیم کنید:

```
git config user.name "Your Name"
git config user.email "your-email@example.com"
```

برای بررسی تنظیمات:

```
git config --list
```

بخش ۶: ساخت README اولیه

یک فایل به نام README.md بسازید و داخل آن فقط این متن را قرار دهید:

Karvand Manager

This repository is for the Karvand Python JSON project.

بخش ۷: ثبت اولین تغییرات با Git

وضعیت پروژه را بررسی کنید:

```
git status -sb
```

فایل README را اضافه کنید:

```
git add README.md
```

تغییرات آماده‌شده را بررسی کنید:

```
git diff --staged
```

سپس اولین Commit را ثبت کنید:

```
git commit -m "docs: add initial README"
```

در پایان تغییرات را به GitHub ارسال کنید:

```
git push origin main
```

تمرین ۲: ساخت پروژه پایتونی مدیریت کاروندها با فایل JSON در Branch جدید

دانشجویان محترم کاروند،

در ادامه‌ی تمرین ۱، داخل همان ریپازیتوری `karvand-manager` یک پروژه پایتونی بسازید که اطلاعات کاروندها را در فایل JSON ذخیره و مدیریت کند.

در این تمرین نباید مستقیم روی شاخه `main` کار کنید. ابتدا یک Branch جدید بسازید و پروژه را داخل آن انجام دهید.

بخش ۱: ساخت Branch جدید

ابتدا مطمئن شوید داخل پوشه پروژه هستید:

```
cd karvand-manager
```

سپس یک Branch جدید بسازید و وارد آن شوید:

```
git switch -c json-manager
```

برای اطمینان از Branch فعلی:

```
git branch
```

بخش ۲: ساخت پروژه پایتونی

ساختار پروژه باید به شکل زیر باشد:

```
karvand-manager/  
├─ app.py  
├─ README.md  
└─ data/  
    └─ karvands.json
```

فایل karvands.json نباید دستی پر شود.
برنامه باید خودش فایل را ایجاد کند و همه اطلاعات فقط از طریق اجرای برنامه وارد، ویرایش یا حذف شوند.

منوی برنامه

برنامه باید یک منوی ساده داشته باشد:

1. افزودن کاروند
2. نمایش کاروندها
3. ویرایش کاروند
4. حذف کاروند
5. گزارش کلی
6. خروج

اطلاعات هر کاروند

برای هر کاروند این اطلاعات ذخیره شود:

- نام کامل
- ایمیل
- شهر
- تحصیلات
- مهارت‌ها

ساختار داده باید JSON تو در تو باشد.
نمونه ساختار هر کاروند:

```
Small JSON Sample
Nested structure for Karvand project

{
  "bootcamp": {
    "title": "Karvand Python",
    "year": 2026
  },
  "karvands": [
    {
      "id": 1,
      "full_name": "Ali Ahmadi",
      "city": "Tehran",
      "education": {
        "degree": "Bachelor",
        "field": "Computer Engineering"
      },
      "skills": [
        {
          "name": "Python",
          "level": "beginner",
          "score": 70
        }
      ]
    }
  ]
}
```

Sample only - JSON must be generated by Python app.

این فقط نمونه ساختار است و نباید دستی داخل فایل JSON نوشته شود.

امکانات برنامه

برنامه باید بتواند:

- کاروند جدید اضافه کند
- فهرست کاروندها را نمایش دهد
- اطلاعات یک کاروند را ویرایش کند
- یک کاروند را حذف کند
- گزارش کلی نمایش دهد

گزارش کلی شامل این موارد باشد:

- تعداد کل کاروندها

فایل README

فایل README.md را کامل تر کنید و این موارد را در آن بنویسید:

- نام پروژه
- توضیح کوتاه درباره پروژه
- امکانات برنامه

بخش ۳: بررسی و ثبت تغییرات

بعد از تکمیل پروژه، وضعیت تغییرات را بررسی کنید:

`git status -sb`

تفاوت تغییرات را ببینید:

`git diff`

فایل‌ها را اضافه کنید:

```
git add app.py README.md data/karvands.json
```

تغییرات آماده‌شده برای Commit را بررسی کنید:

```
git diff --staged
```

سپس Commit بزنید:

```
git commit -m "feat: add karvand json manager project"
```

بخش ۴: ارسال Branch به GitHub

بعد از Commit کردن پروژه، Branch خود را به GitHub ارسال کنید:

```
git push -u origin json-manager
```

بخش ۵: ادغام Branch پروژه با شاخه اصلی

در این مرحله باید پروژه‌ای را که روی Branch ساخته‌اید، وارد شاخه اصلی main کنید تا تمرین‌های بعدی روی نسخه کامل پروژه انجام شوند.

ابتدا به شاخه main برگردید:

```
git switch main
```

آخرین تغییرات شاخه اصلی را از GitHub دریافت کنید:

```
git pull
```

حالا پروژه را با Branch main ادغام کنید:

```
git merge json-manager
```

شاخه main را به GitHub ارسال کنید:

```
git push origin main
```

برای بررسی تاریخچه، دستور زیر را اجرا کنید:

```
git log --oneline --graph --all
```

خروجی مورد انتظار تمرین ۲

برای تحویل تمرین، موارد زیر را ارسال کنید:

۱. خروجی دستور زیر:

```
git log --oneline --graph --all
```

۲. توضیح کوتاه درباره اینکه Branch ساخته شده چه نقشی در انجام تمرین داشته است.

تمرین ۳: ایجاد و حل Conflict در Git

در این تمرین، روی همان پروژه karvand-manager کار می‌کنید.

پیش‌نیاز این تمرین این است که تمرین ۲ را کامل انجام داده باشید و پروژه پایتونی شما روی شاخه main قرار گرفته باشد.

هدف این تمرین این است که بفهمید Conflict در Git چه زمانی اتفاق می‌افتد و چگونه باید آن را حل کرد.

توضیح سناریو

وقتی بین branch ها جابه‌جا می‌شوید، Git فایل‌های پروژه را مطابق وضعیت همان branch نمایش می‌دهد.

یعنی اگر یک خط را روی یک branch تغییر دهید و commit کنید، وقتی به branch دیگر برگردید، آن تغییر را نمی‌بینید؛ چون آن تغییر فقط روی branch قبلی ثبت شده است.

Conflict زمانی ایجاد می‌شود که یک بخش مشترک از یک فایل، در دو branch مختلف، به دو شکل متفاوت تغییر کند. در این تمرین، یک خط مشخص در فایل app.py را در دو branch متفاوت تغییر می‌دهیم:

- یک بار در branch update-title
- یک بار در branch main

سپس هنگام merge کردن، Git نمی‌تواند خودش تصمیم بگیرد کدام تغییر درست است. در نتیجه conflict ایجاد می‌شود.

بخش ۱: آماده‌سازی پروژه

ابتدا وارد پوشه پروژه شوید:

```
cd karvand-manager
```

به شاخه main بروید:

git switch main

آخرین تغییرات را از GitHub دریافت کنید:

git pull

بخش ۲: ایجاد خط پایه روی main

فایل app.py را باز کنید.

بالای فایل اضافه کنید:

```
APP_TITLE = "Karvand Manager"
```

سپس تغییر پایه را روی شاخه main ثبت کنید:

```
git add app.py
```

```
git commit -m "feat: add app title"
```

بخش ۳: ساخت Branch جدید از روی main

حالا از روی وضعیت فعلی main یک Branch جدید بسازید:

```
git switch -c update-title
```

برای اطمینان از اینکه روی Branch جدید هستید، دستور زیر را اجرا کنید:

```
git branch
```

بخش ۴: تغییر عنوان برنامه روی Branch جدید

در Branch update-title ، خط زیر را در فایل app.py پیدا کنید:

```
APP_TITLE = "Karvand Manager"
```

آن را به این مقدار تغییر دهید:

```
APP_TITLE = "Karvand Student Manager"
```

تغییرات را بررسی کنید:

```
git diff
```

سپس تغییر را ثبت کنید:

```
git add app.py  
git commit -m "feat: update app title in update-title branch"
```

بخش ۵: تغییر همان خط روی main

اکنون به شاخه main برگردید:

```
git switch main
```

نکته مهم:

وقتی به main برمی گردید، Git فایل ها را مطابق وضعیت شاخه main نمایش می دهد. بنابراین تغییر Karvand Student Manager که در branch update-title ثبت کردید، در این شاخه دیده نمی شود.

در فایل app.py همان خط را پیدا کنید:

```
APP_TITLE = "Karvand Manager"
```

حالا همان خط را در شاخه main به مقدار متفاوتی تغییر دهید:

```
APP_TITLE = "Karvand JSON Manager"
```

تغییرات را بررسی کنید:

```
git diff
```

سپس تغییر را ثبت کنید:

```
git add app.py  
git commit -m "feat: update app title in main branch"
```

بخش ۶: ایجاد Conflict هنگام Merge

حالا Branch update-title را با شاخه main ادغام کنید:

```
git merge update-title
```

در این مرحله باید conflict ایجاد شود، چون یک خط مشخص از فایل app.py در دو Branch به دو شکل متفاوت تغییر کرده است.

بخش ۷: مشاهده Conflict در فایل

فایل app.py را باز کنید.

بخشی شبیه نمونه زیر مشاهده می کنید:

```
<<<<<<< HEAD
APP_TITLE = "Karvand JSON Manager"
=====
APP_TITLE = "Karvand Student Manager"
>>>>>>> update-title
```

معنی این بخش:

- قسمت HEAD تغییرات شاخه فعلی یعنی main را نشان می دهد.
- قسمت بعد از ===== تغییرات Branch دیگر یعنی update-title را نشان می دهد.
- Git نمی تواند خودش انتخاب کند کدام نسخه درست است.

بخش ۸: حل Conflict

علامت های conflict را حذف کنید:

<<<<<<< HEAD

=====

>>>>>>> update-title

سپس یک مقدار نهایی برای APP_TITLE انتخاب کنید.

برای مثال:

```
APP_TITLE = "Karvand JSON Manager - Student Panel"
```

فایل اصلاح شده را Stage کنید:

```
git add app.py
```

سپس حل conflict را ثبت کنید:

```
git commit -m "fix: resolve app title conflict"
```

بخش ۹: بررسی تاریخچه و ارسال تغییرات

برای مشاهده تاریخچه Branch ها و Merge ، دستور زیر را اجرا کنید:

```
git log --oneline --graph --all
```

در پایان تغییرات شاخه main را به GitHub ارسال کنید:

```
git push origin main
```

۲. خروجی دستور زیر:

```
git log --oneline --graph --all
```

۵. توضیح کوتاه درباره اینکه چرا conflict ایجاد شد.

سؤال تشریحی

چرا Git نتوانست خودش تغییرات دو Branch را ادغام کند؟

تمرین ۴: استفاده از **gitignore**. و برگشت دادن تغییر اشتباه

در این تمرین، روی همان پروژه karvand-manager کار می‌کنید.

هدف این تمرین این است که یاد بگیرید:

- چگونه یک فایل اضافی را از Git خارج نگه دارید
- فایل **gitignore** چه کاربردی دارد
- چگونه یک تغییر اشتباه را با **git restore** برگردانید

بخش ۱: آماده‌سازی پروژه

ابتدا وارد پوشه پروژه شوید:

```
cd karvand-manager
```

به شاخه **main** بروید:

```
git switch main
```

آخرین تغییرات را از GitHub دریافت کنید:

```
git pull
```

بخش ۲: ساخت Branch جدید

برای این تمرین یک Branch جدید بسازید:

```
git switch -c cleanup-project
```

برای اطمینان از Branch فعلی:

git branch

بخش ۳: ساخت فایل اضافی

در ریشه پروژه یک فایل به نام debug.log بسازید.

داخل فایل debug.log این متن را قرار دهید:

Program started...

حالا وضعیت پروژه را بررسی کنید:

git status -sb

در این مرحله Git فایل debug.log را به عنوان یک فایل جدید نشان می دهد.

بخش ۴: ساخت فایل .gitignore

در ریشه پروژه یک فایل به نام .gitignore بسازید.

داخل آن فقط این مورد را قرار دهید:

debug.log

دوباره وضعیت پروژه را بررسی کنید:

git status -sb

اکنون فایل debug.log نباید در خروجی Git نمایش داده شود.

بخش ۵: ثبت قانون نادیده گرفتن فایل debug.log

فقط فایل .gitignore را به Git اضافه کنید:

```
git add .gitignore
```

تغییرات آماده شده را بررسی کنید:

```
git diff --staged
```

سپس Commit بزنید:

```
git commit -m "chore: ignore debug log file"
```

نتیجه این بخش

در این مرحله، فایل `.gitignore` وارد Git می شود، اما فایل `debug.log` وارد Git نمی شود.

دلیل آن این است که داخل `.gitignore` نوشته ایم:

```
debug.log
```

بنابراین Git از این به بعد فایل `debug.log` را نادیده می گیرد و آن را برای `commit` پیشنهاد نمی دهد.

بخش ۶: ایجاد یک تغییر اشتباه

فایل `app.py` را باز کنید و یک خط اشتباه به انتهای آن اضافه کنید:

```
print("wrong test line")
```

حالا تغییرات را بررسی کنید:

```
git diff
```

در خروجی `git diff` باید ببینید که این خط به فایل `app.py` اضافه شده است.

بخش ۷: برگشت دادن تغییر اشتباه

چون این تغییر اشتباه است و نمی‌خواهیم آن را نگه داریم، آن را با دستور زیر برگردانید:

```
git restore app.py
```

دوباره وضعیت پروژه را بررسی کنید:

```
git status -sb
```

اگر تغییر اشتباه درست برگشته باشد، فایل `app.py` دیگر نباید در خروجی `Git` به عنوان فایل تغییر کرده نمایش داده شود.

بخش ۸: ارسال Branch به GitHub

تمرین را به `GitHub` ارسال کنید:

```
git push -u origin cleanup-project
```

در ۲ تا ۳ خط توضیح دهید:

فایل `debug.log` چه نوع فایلی است و چرا بهتر است وارد `Git` نشود؟

تمرین ۵: جست و جوی کاروند در JSON و استفاده از git stash

در این تمرین، روی همان پروژه karvand-manager کار می کنید.

در تمرین های قبلی، برنامه شما اطلاعات کاروندها را داخل فایل JSON ذخیره می کرد. حالا می خواهیم یک قابلیت کاربردی به برنامه اضافه کنیم.

سناریوی تمرین

فرض کنید در بوت کمپ، چند کاروند در فایل JSON ثبت شده اند.

گاهی مسئول کلاس می خواهد خیلی سریع اطلاعات یک کاروند خاص را پیدا کند. برای مثال می گوید:

اطلاعات کاروند شماره ۲ را نشان بده.

پس برنامه باید بتواند از کاربر یک id بگیرد، داخل فایل data/karvands.json بگردد و اگر کاروند مورد نظر وجود داشت، اطلاعاتش را نمایش دهد.

حالا فرض کنید در حال اضافه کردن این قابلیت هستید، اما هنوز کد شما کامل نشده است. بنابراین نمی خواهید commit بزنید، چون تغییرتان هنوز ناقص است.

از طرفی لازم است همین الان برنامه را با نسخه قبلی و سالم اجرا کنید؛ اما تغییراتی که تازه اضافه کرده اید هنوز آماده استفاده نیستند و ممکن است اجرای برنامه را به هم بزنند.

در این حالت از git stash استفاده می کنید تا تغییرات نیمه کاره موقتاً کنار گذاشته شوند. بعد از آن می توانید نسخه سالم برنامه را اجرا کنید و سپس دوباره تغییرات نیمه کاره را برگردانید و کار را ادامه دهید.

هدف تمرین

در پایان این تمرین، برنامه باید بتواند:

۱. فایل data/karvands.json را بخواند.

۲. از کاربر شناسه کاروند را بگیرد.
 ۳. داخل لیست کاروندها جست‌وجو کند.
 ۴. اگر کاروند پیدا شد، اطلاعات او را نمایش دهد.
 ۵. اگر کاروند پیدا نشد، پیام مناسب نمایش دهد.
-

بخش ۱: آماده‌سازی پروژه

ابتدا وارد پوشه پروژه شوید:

```
cd karvand-manager
```

به شاخه main بروید:

```
git switch main
```

آخرین تغییرات را از GitHub دریافت کنید:

```
git pull.
```

بخش ۲: ساخت Branch جدید

برای این تمرین یک Branch جدید بسازید:

```
git switch -c search-karvand
```

برای اطمینان از Branch فعلی، دستور زیر را اجرا کنید:

```
git branch
```

بخش ۳: شروع قابلیت جست‌وجو

فایل app.py را باز کنید.

یک گزینه جدید به منوی برنامه اضافه کنید:

```
جستوجوی کاروند با id  
def search_karvand_by_id():  
    pass
```

فعلاً لازم نیست تابع را کامل کنید.

تغییرات را ببینید:

```
git diff
```

بخش ۴: ذخیره موقت تغییرات با **git stash**

در این لحظه کد شما هنوز کامل نیست. پس نباید **commit** بزنید.

تغییرات نیمه‌کاره را موقتاً ذخیره کنید:

```
git stash push -m "start search karvand feature"
```

وضعیت پروژه را بررسی کنید:

لیست **stash** ها را ببینید:

```
git stash list
```

نتیجه این بخش

شما تغییرات ناقص را **commit** نکردید، اما آن‌ها را هم از دست ندادید.

Git تغییرات را موقتاً در **stash** نگه داشته است تا بعداً بتوانید دوباره آن‌ها را برگردانید.

حالا برای اینکه ببینید نسخه سالم قبلی برنامه هنوز درست اجرا می‌شود، برنامه را اجرا کنید:

```
python app.py
```

در این مرحله فقط برنامه را اجرا و منوی اصلی آن را بررسی کنید. فعلاً تغییر جدیدی در اطلاعات کارونها ایجاد نکنید.

بخش ۵: برگرداندن تغییرات نیمه کاره

حالا تغییراتی را که stash کرده بودید، برگردانید:

```
git stash pop
```

وضعیت پروژه را بررسی کنید:

```
git status -sb
```

اکنون تغییرات نیمه کاره باید دوباره داخل app.py برگشته باشند.

حالا کد را کامل کرده و تابع جستجوی کاروند با id را کامل کنید

بخش ۶: ثبت تغییرات

تغییرات را ببینید:

```
git diff
```

فایل تغییر کرده را اضافه کنید:

```
git add app.py
```

تغییرات آماده شده را بررسی کنید:

```
git diff --staged
```

سپس commit بزنید:

```
git commit -m "feat: add karvand search by id"
```

بخش ۷: ارسال Branch به GitHub

Branch تمرین را به GitHub ارسال کنید:

```
git push -u origin search-karvand
```

سؤال کوتاه

در ۲ تا ۳ خط توضیح دهید:

چرا در این تمرین قبل از کامل شدن کد، از git stash استفاده کردیم و commit نزدیم؟